



 @azdocuments

PROGRAMMING IN C

SUBJECT CODE: 1BEIT105/205

MODULE-2

Name:

USN:

College:

AZ Documents

Your Engineering Study Partner

www.azdocuments.in

PROGRAMMING IN C

1BEIT105/205

MODULE-02

Console I/O: Reading and Writing Characters, Reading and Writing Strings, Formatted Console I/O, printf(), scanf().

Statements: True and False in C, Selection Statements, Iteration Statements, Jump Statements, Expression Statements, Block Statements.

Textbook 1: Chapter 8, 3

CONSOLE I/O: READING AND WRITING CHARACTERS IN C

In C programming, **Console Input/Output (I/O)** refers to the process of reading data from the keyboard and displaying output on the screen.

The simplest console I/O functions for character input and output are:

- getchar() → reads a character from the keyboard
- putchar() → writes a character to the screen

These functions are defined in the **standard input-output header file:**

```
#include <stdio.h>
```

They are commonly used for **character-based input and output operations.**

getchar() Function

The getchar() function is used to **read a single character from the keyboard.**

It waits until the user presses a key and then returns that character.

Syntax

```
int getchar(void);
```

Explanation

- The function **takes no arguments.**
- It reads a character from **standard input (keyboard).**
- The character is **echoed automatically to the screen.**
- It returns the **ASCII value of the character.**

Although the return type is int, the returned value is usually stored in a **char variable** because the actual character is stored in the **low-order byte**.

Example Program

```
#include <stdio.h>
int main()
{
    char ch;

    printf("Enter a character: ");
    ch = getchar();

    printf("You entered: ");
    putchar(ch);

    return 0;
}
```

Output

Enter a character: A
You entered: A

Return Value of getchar()

The getchar() function returns:

Return Value	Meaning
Character value	Character entered
EOF	Error or end-of-file

EOF stands for **End Of File**.

Example:

```
if(ch == EOF)
    printf("Input Error");
```

EOF is defined in:

```
#include <stdio.h>
```

Usually its value is:

-1

putchar() Function

The putchar() function is used to **display a single character on the screen**.

It writes the character to the **current cursor position** on the console.

Syntax

```
int putchar(int c);
```

Explanation

- c represents the character to be printed.
- Even though the parameter type is int, it usually receives a **character**.
- Only the **low-order byte** of the integer is printed.

Example Program

```
#include <stdio.h>
```

```
int main()
{
    char ch = 'A';

    putchar(ch);

    return 0;
}
```

Output

A

Return Value of putchar()

putchar() returns:

Return Value	Meaning
--------------	---------

Character printed	Successful output
-------------------	-------------------

EOF	Error
-----	-------

Example Program Using getchar() and putchar()

The following program reads characters and converts:

- lowercase → uppercase
- uppercase → lowercase

The program stops when a **period (.)** is entered.

Program

```
#include <stdio.h>
#include <ctype.h>
int main(void)
{
    char ch;

    printf("Enter some text (type a period to quit).\n");

    do
    {
        ch = getchar();

        if(islower(ch))
            ch = toupper(ch);
        else
            ch = tolower(ch);

        putchar(ch);
    } while(ch != '.');

    return 0;
}
```



Explanation

Step 1

The program prompts the user to enter text.

Step 2

getchar() reads characters one by one.

Step 3

The function islower() checks if the character is lowercase.

Step 4

If lowercase → convert using toupper().

Step 5

Otherwise convert using `tolower()`.

Step 6

`putchar()` prints the converted character.

Step 7

The loop stops when `.` is entered.

Problem with `getchar()`

One major issue with `getchar()` is **line buffering**.

Line Buffering

Many compilers implement `getchar()` in such a way that:

- Input is stored in a **buffer**
- Characters are returned **only after pressing ENTER**

Example:

User types: Hello

Press ENTER

Only then will `getchar()` start reading characters.

Issues Caused by Line Buffering

1. Program waits until **ENTER is pressed**.
2. Extra characters may remain in the **input buffer**.
3. Not suitable for **interactive programs** like games.

Alternatives to `getchar()`

Because of the limitations of `getchar()`, many compilers provide alternative functions.

The most common ones are:

`getch()`

`getche()`

These functions are usually declared in:

`#include <conio.h>`

getch() Function

getch() reads a character directly from the keyboard **without waiting for ENTER**.

Syntax

```
int getch(void);
```

Characteristics

Feature	Description
Reads one character	Yes
Waits for ENTER	No
Echo to screen	No
Immediate input	Yes

Example

```
#include <stdio.h>
#include <conio.h>
int main()
{
    char ch;

    printf("Press any key: ");
    ch = getch();

    printf("\nYou pressed %c", ch);

    return 0;
}
```

getche() Function

getche() works like getch(), but the character **is displayed on the screen**.

Syntax

```
int getche(void);
```

Characteristics

Feature	Description
---------	-------------

Reads character	Yes
-----------------	-----

Waits for ENTER	No
-----------------	----

Echo character	Yes
----------------	-----

Immediate input	Yes
-----------------	-----

Example Using getch()

```
#include <stdio.h>
#include <conio.h>
#include <ctype.h>
int main()
{
    char ch;
    printf("Enter text (type . to quit):\n");
    do
    {
        ch = getch();

        if(islower(ch))
            ch = toupper(ch);
        else
            ch = tolower(ch);

        putchar(ch);

    } while(ch != '.');

    return 0;
}
```

Difference Between getchar(), getch(), and getche()

Function Waits for ENTER Echo Character

getchar()	Yes	Yes
-----------	-----	-----

getch()	No	No
---------	----	----

getche()	No	Yes
----------	----	-----

Character-based console input and output in C is primarily performed using `getchar()` and `putchar()` functions. The `getchar()` function reads a single character from the keyboard, while `putchar()` displays a character on the screen. However, because `getchar()` often uses line buffering, functions such as `getch()` and `getche()` are commonly used for interactive input. Understanding these functions is essential for handling character input and output operations in C programs.

READING AND WRITING STRINGS IN C

In C programming, **console input and output functions** allow programs to interact with users through the keyboard and display results on the screen.

After character I/O functions like:

- `getchar()` – reads a character
- `putchar()` – prints a character

the next level of console I/O functions handle **strings**.

The two main functions used for **reading and writing strings** are:

- `gets()` – reads a string from the keyboard
- `puts()` – writes a string to the screen

These functions are defined in the **standard input-output header file**:

```
#include <stdio.h>
```

1. `gets()` Function

The `gets()` function is used to **read a string from the keyboard** and store it in a character array.

The user can type characters until **ENTER (carriage return)** is pressed.

Syntax

```
char *gets(char *str);
```

Explanation

- `str` is a **pointer to a character array** where the input string will be stored.
- Characters are read from the keyboard and stored in the array.
- The input ends when the **ENTER key is pressed**.
- The ENTER key is **not stored in the string**.
- Instead, a **null character (\0)** is added at the end of the string.

Thus the string becomes a **null-terminated string**.

Example Program

```
#include <stdio.h>
int main()
{
    char name[50];
    printf("Enter your name: ");
    gets(name);
    printf("Your name is: %s", name);

    return 0;
}
```

Output

Enter your name: Iqra
Your name is: Iqra

How gets() Works

Suppose the user enters:

Hello

The array will store:

H e l l o \0

The **null character (\0)** marks the end of the string.

Example Program: Finding Length of String

```
#include <stdio.h>
#include <string.h>
int main(void)
{
    char str[80];
    printf("Enter a string: ");
    gets(str);

    printf("Length is %d", strlen(str));
    return 0;
}
```

Output

Enter a string: Programming
Length is 11

Problem with gets()

One major problem with gets() is that it **does not check the size of the array**.

This means the user can enter more characters than the array can hold.

This leads to **buffer overflow**.

Example Problem

```
char str[10];  
gets(str);
```

If the user enters:

ProgrammingLanguage

The string will **overflow the array**, causing:

- memory corruption
- program crash
- security vulnerabilities

Why gets() is Dangerous

Because gets() performs **no boundary checking**, it is considered **unsafe**.

For this reason, it is **not recommended for real-world or commercial programs**.

Instead, programmers use:

fgets()

which allows the programmer to specify the **maximum number of characters** to read.

2. puts() Function

The puts() function is used to **display a string on the screen**.

It prints the string followed by a **newline character automatically**.

Syntax

```
int puts(const char *str);
```

Explanation

- str is the string to be displayed.
- puts() writes the string to the console.
- After printing the string, it automatically moves the cursor to the **next line**.

Example Program

```
#include <stdio.h>
int main()
{
    puts("Hello");
    puts("Welcome to C Programming");
    return 0;
}
```

Output

Hello
Welcome to C Programming

Example Using gets() and puts()

```
#include <stdio.h>

int main()
{
    char str[100];
    puts("Enter a string:");
    gets(str);
    puts("You entered:");
    puts(str);
    return 0;
}
```

Output

Enter a string:
C Programming
You entered:
C Programming

Features of puts()

- Displays a **string on the screen**
- Automatically adds a **newline**
- Faster than printf()
- Simpler syntax

puts() vs printf()

Feature	puts() printf()	
Prints string	Yes	Yes
Prints numbers	No	Yes
Format conversion	No	Yes
Automatically adds newline	Yes	No
Speed	Faster	Slower

Example:

```
puts("Hello");
```

Equivalent printf statement:

```
printf("Hello\n");
```

Return Value of puts()

The puts() function returns:

Value	Meaning
Non-negative value	Success
EOF	Error

However, when printing to the console, errors are rare, so the return value is usually **ignored**.

Example

```
puts("Hello");
```

Output

Hello

Function Operation

getchar() Reads a character from keyboard

getch() Reads a character without echo

getche() Reads a character with echo

putchar() Writes a character to screen

gets() Reads a string

Function Operation

puts() Writes a string

Example Program: Simple Dictionary

The following program demonstrates string input and output functions.

It asks the user to enter a word and displays its meaning.

Program

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
char *dic[][2] = {
    "atlas", "A volume of maps.",
    "car", "A motorized vehicle.",
    "telephone", "A communication device.",
    "airplane", "A flying machine.",
    "", ""
};
int main()
{
    char word[80];
    char **p;
    printf("Enter word: ");
    scanf("%s", word);
    p = (char **)dic;
    while(*p)
    {
        if(!strcmp(*p, word))
        {
            printf("Meaning: %s", *(p+1));
            break;
        }

        p = p + 2;
    }
    if(!*p)
        printf("Word not in dictionary.");

    return 0;
}
```

Explanation of Dictionary Program

1. The program stores words and meanings in a **two-dimensional array of pointers**.
2. The user enters a word.
3. The program searches the array using `strcmp()`.
4. If the word is found, the meaning is printed using `puts()` or `printf()`.
5. If the word is not found, it displays:

Word not in dictionary

Reading and writing strings is an important part of console input and output in C programming. The `gets()` function reads a string from the keyboard, while `puts()` displays a string on the screen. Although these functions are simple and efficient, `gets()` is unsafe because it does not check array boundaries. Therefore, safer alternatives such as `fgets()` are preferred in modern C programming. Understanding these functions is essential for handling text input and output in C programs.

FORMATTED CONSOLE INPUT AND OUTPUT IN C

In C programming, **formatted input and output functions** allow programs to read and display data in **specific formats controlled by the programmer**.

The two most important formatted console I/O functions are:

- **printf()** – prints formatted output to the screen
- **scanf()** – reads formatted input from the keyboard

These functions are defined in the header file:

```
#include <stdio.h>
```

Unlike simple I/O functions (`getchar`, `putchar`, `gets`, `puts`), formatted I/O functions allow the programmer to **control how data is displayed or read** using **format specifiers**.

printf() Function

The **printf()** function is used to **display formatted output on the console**.

It allows printing of:

- characters
- integers
- floating numbers

- strings
- addresses
- formatted tables

Syntax

```
int printf(const char *control_string, ...);
```

Explanation

Control String

The **control string** contains two types of items:

Normal characters

These characters are printed directly.

Example

```
printf("Hello World");
```

Output

Hello World

Format Specifiers

Format specifiers **begin** with % and indicate how the data should be printed.

Example

```
printf("Age = %d", age);
```

Return Value

printf() returns:

- number of characters printed
- negative value if an error occurs

Example of printf()

```
#include <stdio.h>
int main()
{
    printf("I like %c %s", 'C', "very much!");
    return 0;
}
```

Output

I like C very much!

Explanation

Specifier Argument

%c 'C'

%s "very much!"

Arguments are matched **left to right**.

printf() Format Specifiers

Specifier Meaning

%c Character

%d Signed integer

%i Signed integer

%u Unsigned integer

%f Floating point number

%e Scientific notation

%E Scientific notation (uppercase)

%g Shortest representation of %f or %e

%G Same as %g but uppercase

%o Octal number

%x Hexadecimal lowercase

%X Hexadecimal uppercase

%s String

%p Pointer address

%n Stores number of characters printed

%% Prints % symbol

Printing Characters

To print a character use %c.

Example

```
#include <stdio.h>
int main()
{
    char ch = 'A';
    printf("Character: %c", ch);
}
```

Output

Character: A

Printing Strings

To print strings use %s.

Example

```
#include <stdio.h>

int main()
{
    char name[] = "xyz";
    printf("Name: %s", name);
}
```

Output

Name: xyz

Printing Numbers

Integer

```
printf("%d", 100);
```

Output

100

Unsigned Integer

```
printf("%u", 300);
```

Floating Point

```
printf("%f", 10.5);
```

Output

10.500000

Scientific Notation

Scientific notation prints numbers like:

x.dddddE±yy

Example

```
printf("%e", 12345.6);
```

Output

1.234560e+04

Using %g Specifier

%g automatically chooses the shorter format between %f and %e.

Example

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    double f;
```

```
    for(f=1.0; f<1.0e+10; f=f*10)
```

```
        printf("%g ", f);
```

```
}
```

Output

1 10 100 1000 10000 100000 1e+006 1e+007

Octal and Hexadecimal Numbers

Octal

```
printf("%o", 10);
```

Output

12

Hexadecimal

```
printf("%x", 255);
```

Output

ff

Uppercase

```
printf("%X", 255);
```

Output

FF

Example Program

```
#include <stdio.h>
int main()
{
    unsigned num;
    for(num=0; num<16; num++)
    {
        printf("%o ", num);
        printf("%x ", num);
        printf("%X\n", num);
    }

    return 0;
}
```

Output

0 0 0

1 1 1

2 2 2

...

Displaying Address (%p)

To print the **memory address of a variable**, use %p.

Example

```
#include <stdio.h>
int sample;
int main()
{
    printf("%p", &sample);
}
```

Output

0061FF18

%n Specifier

%n stores the number of characters printed so far.

Example

```
#include <stdio.h>
int main()
{
    int count;
    printf("this%n is a test\n", &count);
    printf("%d", count);
}
```

Output

this is a test
4

Explanation

- %n stores the number of characters printed before it.
- "this" contains **4 characters**.

Format Modifiers

Format modifiers allow the programmer to control:

- field width
- decimal precision
- justification

Minimum Field Width

Syntax

`%widthspecifier`

Example

```
printf("%10d", 100);
```

Output

100

Spaces are added to fill width.

Zero Padding

```
printf("%05d", 23);
```

Output

00023

Example Program

```
#include <stdio.h>
int main()
{
    double item = 10.12304;
    printf("%f\n", item);
    printf("%10f\n", item);
    printf("%012f\n", item);

    return 0;
}
```

Output

```
10.123040
10.123040
00010.123040
```

Using printf() for Tables

Example program showing squares and cubes.

```
#include <stdio.h>
int main()
{
    int i;
```

```
for(i=1;i<20;i++)  
    printf("%8d %8d %8d\n", i, i*i, i*i*i);  
return 0;  
}
```

Output

1	1	1
2	4	8
3	9	27

Columns are aligned using %8d.

Precision Specifier

Precision is written as:

%.*number*

Example

```
printf("%.4f", 123.123456);
```

Output

123.1235

Precision with Strings

Example

```
printf("%10.15s", "This is a simple test");
```

Output

This is a simpl

The string is truncated.

Precision with Integers

Example

```
printf("%3.8d", 1000);
```

Output

00001000

Left Justification

By default output is **right justified**.

To left justify use -.

Example

```
printf("%-8d", 100);
```

Example Program

```
#include <stdio.h>
int main()
{
    printf(".....\n");
    printf("right-justified: %8d\n", 100);
    printf("left-justified: %-8d\n", 100);

    return 0;
}
```

Output

```
.....
right-justified:   100
left-justified: 100
```

SCANF() FUNCTION (FORMATTED INPUT)

scanf() reads **formatted input from the keyboard**.

Syntax

```
scanf("control_string", &variable);
```

Example

```
#include <stdio.h>
int main()
{
    int age;
    printf("Enter age: ");
    scanf("%d", &age);
    printf("Age is %d", age);
}
```

Common scanf Specifiers

Specifier Meaning

%d	Integer
%f	Float
%lf	Double
%c	Character
%s	String
%u	Unsigned integer
%x	Hexadecimal

Formatted console I/O in C allows programs to display and read data in **controlled formats**. The `printf()` function is used for formatted output, while `scanf()` is used for formatted input. Using format specifiers and modifiers, programmers can display numbers, strings, characters, addresses, and tables in precise formats. These functions are fundamental for creating user-interactive programs in C.

HANDLING OTHER DATA TYPES IN FORMATTED CONSOLE I/O (PRINTF AND SCANF)

In C programming, the formatted I/O functions **`printf()`** and **`scanf()`** can handle many different types of data such as:

- integers
- floating point numbers
- characters
- strings
- pointers
- short and long integers
- wide characters

To properly handle these different data types, **format modifiers** are used along with format specifiers. These modifiers help the compiler understand the **exact type and size of the data** being processed.

Format Modifiers for `printf()`

Format modifiers are placed **between the percent sign (%) and the format specifier**.

They modify the behavior of format specifiers such as:

%d
%i
%o
%u
%x

Long Integer Modifier (l)

The **l (ell)** modifier tells printf() that the argument is of type **long int**.

Syntax

%ld

Example

```
#include <stdio.h>
int main()
{
    long int num = 123456789;

    printf("Long integer: %ld", num);

    return 0;
}
```

Output

Long integer: 123456789

Short Integer Modifier (h)

The **h modifier** is used when the argument is a **short integer**.

Syntax

%hd

Example

```
#include <stdio.h>
int main()
{
    short int num = 100;
    printf("Short integer: %hd", num);
}
```

```
    return 0;
}
```

Output

Short integer: 100

Using h and l with %n

The %n specifier stores the number of characters printed so far.

Modifiers can be used with %n:

Modifier Meaning

%hn pointer to short int

%ln pointer to long int

Example

```
#include <stdio.h>
int main()
{
    int count;

    printf("Hello%n", &count);
    printf("\nCharacters printed = %d", count);

    return 0;
}
```

Output

Hello
Characters printed = 5

Wide Character Support

Some C compilers support **wide characters**.

Wide characters allow representation of characters from **large character sets** such as Unicode.

Using l with %c and %s:

Format Meaning

%lc wide character

Format Meaning

`%ls` wide character string

Example

```
wchar_t ch;  
printf("%lc", ch);
```

Floating Point Modifier (L)

The **L modifier** is used for **long double** data type.

It can be applied to:

`%e`

`%f`

`%g`

Example

```
#include <stdio.h>  
int main()  
{  
    long double num = 123.456789;  
    printf("%Lf", num);  
    return 0;  
}
```

New Modifiers in C99

C99 introduced two new modifiers:

Modifier Data Type

`hh` signed/unsigned char

`ll` long long integer

hh Modifier

Used with:

`%d %i %o %u %x %n`

Example

```
signed char ch = 10;
printf("%hhd", ch);
```

ll Modifier

Used for **long long integers**.

Example

```
#include <stdio.h>
int main()
{
    long long int num = 1234567890123;

    printf("%lld", num);
    return 0;
}
```

Output

1234567890123

Special printf() Modifiers

Two additional modifiers supported by printf() are:

*

Modifier

The **# modifier** forces special formatting.

For floating numbers

It ensures a **decimal point is printed**.

Example

```
printf("%#f", 10.0);
```

Output

10.000000

For hexadecimal numbers

It adds the prefix 0x.

Example

```
printf("%#x", 10);
```

Output

0xa

For octal numbers

It adds a leading zero.

Example

```
printf("%#o", 10);
```

Output

012

* Modifier

The * modifier allows the **field width and precision to be passed as arguments**.

Example

```
#include <stdio.h>
int main()
{
    printf("%*.*f", 10, 4, 1234.34);
}
```

Explanation

Argument Meaning

10 field width

4 precision

1234.34 value

Output

1234.3400

Example Program Using # and *

```
#include <stdio.h>
int main()
{
    printf("%x %#x\n", 10, 10);
    printf("%*.*f", 10, 4, 1234.34);

    return 0;
}
```

Output

```
a 0xa
1234.3400
```

SCANF() FUNCTION

scanf() is the **general-purpose formatted input function** in C.

It reads data from the **keyboard** and converts it into the required data type.

Syntax

```
int scanf(const char *control_string, ...);
```

Return Value

scanf() returns:

Return Value	Meaning
number of variables successfully assigned	success
EOF	error

Structure of scanf() Control String

The control string contains three types of characters:

1. **Format specifiers**
2. **White-space characters**
3. **Non-white-space characters**

scanf() Format Specifiers

Specifier Meaning

%c	read character
%d	decimal integer
%i	integer
%f	float
%e	scientific float
%g	floating number
%o	octal
%x	hexadecimal
%s	string
%p	pointer
%u	unsigned integer
%n	number of characters read
%[]	scanset
%%	read % symbol

Inputting Numbers

Example

```
#include <stdio.h>
int main()
{
    int num;
    scanf("%d", &num);
    printf("Number = %d", num);
    return 0;
}
```

Inputting Octal and Hexadecimal

Example

```
#include <stdio.h>
int main()
{
    int i, j;
    scanf("%o%x", &i, &j);
    printf("%o %x", i, j);
    return 0;
}
```

Inputting Unsigned Integer

```
unsigned num;
scanf("%u", &num);
```

Reading Characters Using scanf()

Example

```
char a, b, c;
scanf("%c%c%c", &a, &b, &c);
```

If input is

x y

Then

a = x

b = space

c = y

Reading Strings Using scanf()

Example

```
char str[80];
scanf("%s", str);
```

Important point:

scanf("%s") stops reading **when a space is encountered**.

Example

Input

hello there

Output

hello

Inputting an Address

The %p specifier reads a **memory address**.

Example

```
char *p;  
scanf("%p", &p);
```

The %n Specifier in scanf()

The %n specifier stores the **number of characters read so far**.

Example

```
int count;  
scanf("%d%n", &num, &count);
```

Scanset in scanf()

A **scanset** allows scanning only certain characters.

Syntax

%[characters]

Example

%[XYZ]

This means only **X, Y, Z characters** are accepted.

Example Program

```
#include <stdio.h>  
int main()  
{  
    int i;  
    char str[80], str2[80];  
    scanf("%d%[abcdefg]%s", &i, str, str2);
```

```
printf("%d %s %s", i, str, str2);  
return 0;  
}
```

Input

123abcdtye

Output

123 abcd tye

Inverted Scanset

Use ^ to accept characters **not in the set**.

Example

```
%[^abc]
```

This reads characters **until a, b, or c is found**.

White Space in scanf()

Whitespace characters include:

- space
- tab
- newline
- vertical tab
- form feed

Whitespace in the control string causes scanf() to **skip leading whitespace in input**.

Non-Whitespace Characters in Control String

Example

```
scanf("%d,%d", &x, &y);
```

Input must contain a comma.

Example

10,20

Passing Addresses to scanf()

All variables passed to scanf() must be **addresses**.

Example

```
scanf("%d", &x);
```

Because C uses **call by reference** here.

Maximum Field Width in scanf()

Example

```
scanf("%20s", str);
```

This reads **maximum 20 characters**.

If input is

ABCDEFGHIJKLMNOPQRSTUVWXYZ

Stored in array

ABCDEFGHIJKLMNOPQRSTUVWXYZ

Remaining characters remain in input buffer.

Suppressing Input

Use * to read but **ignore** input.

Example

```
scanf("%d%*c%d", &x, &y);
```

Input

10,20

The comma is read but ignored.

Handling different data types in formatted console I/O requires the use of **format specifiers and modifiers** in printf() and scanf(). These allow the programmer to precisely control how data is displayed and read. Modifiers such as h, l, L, hh, and ll help specify data sizes, while additional modifiers like # and * enhance formatting flexibility. Understanding these concepts is essential for writing robust and user-friendly C programs.

Statements in C

In C programming, a **statement** is a command that instructs the computer to perform a specific action.

Every program is composed of a series of statements that control the **flow of execution**.

A statement may:

- perform a calculation
- make a decision
- repeat an action
- transfer control to another part of the program

Categories of Statements in C

C classifies statements into six main categories:

1. **Selection statements**
2. **Iteration statements**
3. **Jump statements**
4. **Label statements**
5. **Expression statements**
6. **Block statements**

Each category serves a different purpose in controlling program execution.

1. Selection Statements

Selection statements allow the program to **choose between multiple actions** based on a condition.

They are also called **conditional statements**.

C supports two selection statements:

- if
- switch

Additionally, the **ternary operator (?)** can act as a short form of if.

2. Iteration Statements

Iteration statements allow repeated execution of a block of code.

They are also known as **loop statements**.

C provides three iteration statements:

- while
- for
- do-while

3. Jump Statements

Jump statements transfer control to another part of the program.

They include:

- break
- continue
- goto
- return

4. Label Statements

Label statements are used with **goto and switch statements**.

They include:

- case
- default
- user-defined labels

5. Expression Statements

An **expression statement** consists of an expression followed by a semicolon.

Example:

```
x = y + 5;
```

6. Block Statements

A **block statement** is a group of statements enclosed in curly braces.

Example:

```
{  
    int a = 10;  
    int b = 20;  
    printf("%d", a+b);  
}
```

Blocks are also called **compound statements**.

True and False in C

Many statements rely on **conditional expressions**.

A conditional expression evaluates to either **true** or **false**.

In C:

Value	Meaning
-------	---------

0	False
---	-------

Any non-zero value	True
--------------------	------

Example:

```
if(5)  
    printf("True");
```

Since $5 \neq 0$, the condition is **true**.

Even negative numbers are considered true.

Example

```
if(-10)  
    printf("True");
```

Selection Statements in Detail

The two selection statements are:

1. if
2. switch

The most commonly used is the **if statement**.

The if Statement

The **if statement** is used to execute a block of code only if a specified condition is true.

General Syntax

```
if(expression)
statement;
else
statement;
```

Important points:

- The **else part is optional**
- The expression must produce a **scalar value**

Scalar Values

A scalar value is one of the following types:

- integer
- character
- pointer
- floating point

(C99 also includes `_Bool`.)

However, floating-point conditions are **rarely used** because they require more processing time.

Working of if Statement

1. The expression is evaluated.
2. If the result is **true (non-zero)**, the if statement executes.
3. If the result is **false (0)**, the else statement executes.

Only **one branch executes**, never both.

Example: Magic Number Program

This program checks whether the user guessed the correct number.

Program

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int magic;
    int guess;
    magic = rand();
    printf("Guess the magic number: ");
    scanf("%d", &guess);
    if(guess == magic)
        printf("*** Right ***");
    return 0;
}
```

Explanation

1. rand() generates a random number.
2. The user guesses a number.
3. If both numbers match, the program prints:

Right

if-else Statement

The **else clause** allows an alternative action if the **condition** is false.

Example

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int magic;
    int guess;
    magic = rand();
    printf("Guess the magic number: ");
    scanf("%d", &guess);
    if(guess == magic)
        printf("*** Right ***");
    else
        printf("Wrong");
    return 0;
}
```

Nested if Statements

A **nested if** is an if statement placed inside another if statement.

Nested if statements are used when **multiple conditions must be checked**.

Important Rule of Nested if

An else always belongs to the nearest unmatched if within the same block.

Example

```
if(i)
{
    if(j)
        dosomething1();

    if(k)
        dosomething2();
    else
        dosomething3();
}
else
    dosomething4();
```

Here:

- the inner else belongs to if(k)
- the outer else belongs to if(i)

Nesting Limits

C89 requires at least **15 levels of nesting**.

C99 increases this to **127 levels**.

However, deep nesting makes programs **difficult to read**.

Example Program with Nested if

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int magic;
```

```
int guess;
magic = rand();
printf("Guess the magic number: ");
scanf("%d", &guess);
if(guess == magic)
{
    printf("*** Right ***");
    printf("%d is the magic number\n", magic);
}
else
{
    printf("Wrong, ");
    if(guess > magic)
        printf("Too high\n");
    else
        printf("Too low\n");
}
return 0;
}
```

if-else-if Ladder

When many conditions must be checked sequentially, we use an **if-else-if ladder**.

General Syntax

```
if(condition1)
statement;
else if(condition2)
statement;
else if(condition3)
statement;
else
statement;
```

Working

1. Conditions are evaluated from **top to bottom**.
2. The first **true condition** executes.
3. Remaining conditions are **skipped**.
4. If none are true, the **final else** executes.

Example Program

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int magic;
    int guess;
    magic = rand();
    printf("Guess the magic number: ");
    scanf("%d", &guess);
    if(guess == magic)
    {
        printf("*** Right ***");
        printf("%d is the magic number", magic);
    }
    else if(guess > magic)
        printf("Wrong, too high");
    else
        printf("Wrong, too low");
    return 0;
}
```

The ? (Ternary Operator)

The **? operator** is a shorthand form of an **if-else statement**.

It is called a **ternary operator** because it requires **three operands**.

General Syntax

expression1 ? expression2 : expression3

Working

1. Evaluate **expression1**
2. If true → execute **expression2**
3. If false → execute **expression3**

Example

```
x = 10;
y = x > 9 ? 100 : 200;
```

Here

$x > 9 \rightarrow \text{true}$

So

$y = 100$

Equivalent if-else Statement

```
if(x > 9)
    y = 100;
else
    y = 200;
```

Example Program: Square with Sign Preservation

```
#include <stdio.h>
int main()
{
    int i, isqrd;
    printf("Enter a number: ");
    scanf("%d", &i);

    isqrd = i > 0 ? i*i : -(i*i);
    printf("%d squared is %d", i, isqrd);
    return 0;
}
```

Example with Functions

The ternary operator can also execute functions.

```
#include <stdio.h>
int f1(int n);
int f2(void);
int main()
{
    int t;
    printf("Enter a number: ");
    scanf("%d", &t);
    t ? f1(t) + f2() : printf("zero entered.");
    printf("\n");
}
```

Functions:

```
int f1(int n)
{
    printf("%d ", n);
    return 0;
}
```

```
int f2(void)
{
    printf("entered ");
    return 0;
}
```

Explanation

If the user enters **0**

Output

zero entered

If user enters **5**

Output

5 entered

Important Note

The compiler may **change the order of evaluation** of expressions to optimize code.

Therefore function execution order in **complex expressions** may sometimes vary.

Example: Magic Number Program Using ?

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int magic;
    int guess;
    magic = rand();
    printf("Guess the magic number: ");
    scanf("%d", &guess);
```

```
if(guess == magic)
{
    printf("*** Right ***");
    printf("%d is the magic number", magic);
}
else
    guess > magic ? printf("High") : printf("Low");

return 0;
}
```

Explanation

If $\text{guess} > \text{magic}$

High

If $\text{guess} < \text{magic}$

Low

Statements control the flow of execution in C programs. Selection statements such as if, if-else, nested if, and if-else-if allow programs to make decisions based on conditions. The ternary operator ? provides a compact alternative to simple if-else statements. Understanding these control structures is essential for implementing logical decision-making in C programs.

THE CONDITIONAL EXPRESSION IN C

In C programming, conditional statements such as **if** and the **ternary operator (?)** rely on **expressions that evaluate to true or false**.

One important concept in C is that **any valid expression can control a conditional statement**.

Unlike some languages such as **BASIC or Pascal**, C does **not restrict conditions to relational or logical expressions**.

Instead, the only requirement is:

The expression must evaluate to **true (non-zero)** or **false (0)**.

True and False Values

In C:

Value	Meaning
-------	---------

0	False
---	-------

Any non-zero value	True
--------------------	------

This includes:

- positive numbers
- negative numbers
- characters
- pointers

Example

```
if(5)
printf("True");
```

Since **5 is non-zero**, the condition is **true**.

Example: Avoiding Division by Zero

The following program reads two numbers and divides the first by the second **only if the second number is not zero**.

Program

```
/* Divide the first number by the second */
#include <stdio.h>
int main(void)
{
    int a, b;
    printf("Enter two numbers: ");
    scanf("%d%d", &a, &b);

    if(b)
        printf("%d\n", a/b);
    else
        printf("Cannot divide by zero.\n");

    return 0;
}
```

Explanation

1. The program reads two integers **a** and **b**.
2. The condition of the if statement is simply **b**.
3. If **b** is **non-zero**, the condition is true and division occurs.
4. If **b = 0**, the condition is false and the program prints an error message.

Example

Input

Enter two numbers: 10 2

Output

5

Input

Enter two numbers: 10 0

Output

Cannot divide by zero.

Why Writing `if(b != 0)` is Considered Bad Style

The following code is **redundant**:

```
if(b != 0)
printf("%d", a/b);
```

Because

```
if(b)
```

already checks whether **b** is zero or not.

Therefore:

- `if(b)` is **shorter**
- `if(b)` is **more efficient**
- `if(b)` is **better programming style**

THE SWITCH STATEMENT

C provides a **multiple-branch selection statement** called **switch**.

The switch statement allows a program to select **one block of code from many possible options**.

It is commonly used in:

- menu-driven programs
- keyboard command processing
- decision making with many choices

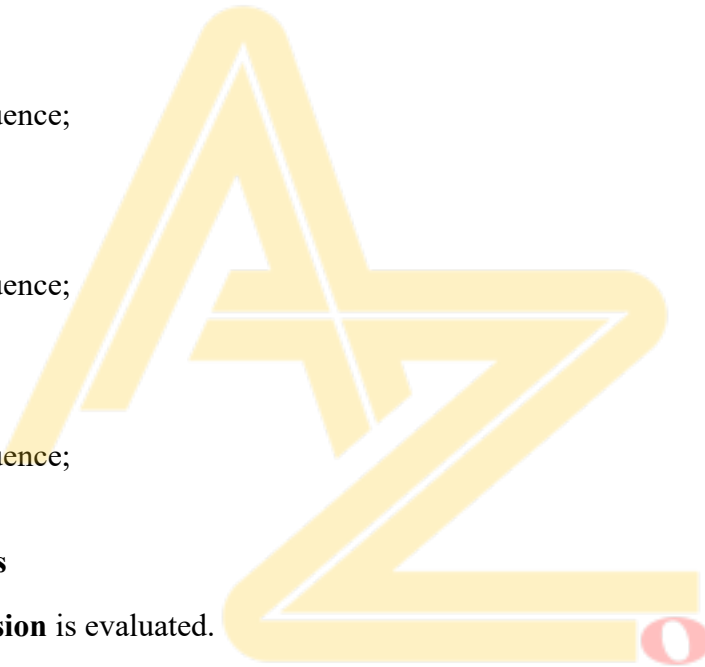
General Syntax of switch

```
switch(expression)
{
    case constant1:
        statement sequence;
        break;

    case constant2:
        statement sequence;
        break;

    case constant3:
        statement sequence;
        break;

    default:
        statement sequence;
}
```



How switch Works

1. The **expression** is evaluated.
2. Its value is compared with each **case constant**.
3. If a match is found:
 - the corresponding statements execute.
4. Execution continues until:
 - a break statement occurs, or
 - the end of the switch block is reached.
5. If no case matches:
 - the **default** block executes.

Important Rules of switch

Expression Must Be Integer Type

The expression must evaluate to:

- integer
- character

Floating-point expressions **cannot be used**.

Example

Valid

switch(x)

Invalid

switch(10.5)

Case Values Must Be Constants

Case labels must be:

- integer constants
- character constants

Example

case 1:

case 2:

case 'A':

Case Values Must Be Unique

Two cases **cannot have the same constant value**.

Invalid example

case 2:

case 2:

default is Optional

If no case matches:

- default executes

If default is missing:

- nothing happens.

break Statement

break terminates the switch block.

Without break, execution continues into the next case.

Example: Menu Program

The following function displays a menu for a spelling checker program.

Program

```
void menu(void)
{
    char ch;
    printf("1. Check Spelling\n");
    printf("2. Correct Spelling Errors\n");
    printf("3. Display Spelling Errors\n");
    printf("Strike Any Other Key to Skip\n");
    printf("Enter your choice: ");
    ch = getchar();
    switch(ch)
    {
        case '1':
            check_spelling();
            break;

        case '2':
            correct_errors();
            break;

        case '3':
            display_errors();
            break;

        default:
            printf("No option selected");
    }
}
```

Explanation

1. The user presses a key.
2. The program reads the key using `getchar()`.
3. The switch statement checks the key.
4. Based on the key pressed:
 - the corresponding function executes.

Example

Input

2

Output

Correct Spelling Errors

Drop-Through Behavior

In a switch statement, if a `break` is missing, execution continues to the next case.

This behavior is called **drop-through**.

Example

```
void inp_handler(int i)
{
    int flag;
    flag = -1;
    switch(i)
    {
        case 1:
        case 2:
        case 3:
            flag = 0;
            break;
        case 4:
            flag = 1;
        case 5:
            error(flag);
            break;
        default:
            process(i);
    }
```

```
}  
}
```

Explanation

Case 1, 2, 3

All three cases share the same statements.

```
flag = 0;  
break;
```

Case 4

If i = 4

```
flag = 1
```

Since there is **no break**, execution **continues** to case 5.

So

```
error(flag)  
is executed.
```

Case 5

If i = 5

```
error(flag)  
is executed directly.
```

Advantages of Drop-Through

Drop-through allows:

- avoiding duplicate code
- reducing program size
- improving efficiency

Nested switch Statements

A switch statement can be placed **inside another switch**.

This is called a **nested switch**.

Example

```
switch(x)
{
    case 1:
        switch(y)
        {
            case 0:
                printf("Divide by zero error\n");
                break;

            case 1:
                process(x, y);
                break;
        }
        break;

    case 2:
        printf("Case 2 executed");
}
```

Explanation

1. The outer switch checks **x**.
2. If $x = 1$, the inner switch checks **y**.
3. Based on the value of **y**, different actions occur.

Important Point

Even if inner and outer switches use the **same case values**, there is **no conflict**.

Difference Between if and switch

Feature	if	switch
Type of conditions	Any relational or logical expression	Only equality
Data types	Any scalar	Integer or char
Complexity	Suitable for complex conditions	Suitable for multiple choices
Execution speed	Slower with many conditions	Faster for many cases

The conditional expression concept in C allows any valid expression to control decision-making statements. Selection statements such as **if and switch** enable a program to choose different execution paths based on conditions. The switch statement is particularly useful when a variable must be compared with multiple constant values. Understanding these concepts is essential for designing efficient decision-making logic in C programs.

ITERATION STATEMENTS IN C

In programming, many problems require **repeating a set of instructions multiple times**. Instead of writing the same instructions again and again, programming languages provide **iteration statements**, also known as **loops**.

An **iteration statement** repeatedly executes a block of code until a specified condition becomes false.

Loops make programs:

- shorter
- easier to maintain
- efficient
- easier to understand

Without loops, many tasks such as **processing arrays, reading data repeatedly, or performing calculations multiple times** would require very long programs.

Iteration statements are control structures that repeatedly execute a statement or block of statements until a specified condition is reached.

Types of Iteration Statements in C

C provides **three looping statements**:

1. **for loop**
2. **while loop**
3. **do-while loop**

Each loop has its own advantages and is used depending on the situation.

Types of Loop Conditions

Loops can work based on two types of conditions:

1 Predetermined loops

The number of iterations is known beforehand.

Example

Print numbers from 1 to 10

Best loop → **for loop**

2 Open-ended loops

The number of iterations is not known.

Example

Keep reading input until user presses A

Best loops → **while loop or do-while loop**

The for Loop

The **for loop** is one of the most commonly used loops in C.

It is especially useful when:

- the number of repetitions is known
- a loop control variable is used
- initialization, condition, and increment must be handled together

The **for loop provides great flexibility and power in C** compared with many other programming languages.

General Syntax

```
for(initialization; condition; increment)
statement;
```

If multiple statements are needed:

```
for(initialization; condition; increment)
{
    statements
}
```

Components of the for Loop

The for loop has **three important parts**.

1 Initialization

The initialization statement sets the **starting value of the loop control variable**.

Example

```
x = 1
```

This executes **only once**, when the loop begins.

2 Condition

The condition determines **whether the loop should continue executing**.

The loop continues **as long as the condition is true**.

Example

```
x <= 100
```

When the condition becomes false, the loop stops.

3 Increment / Update

The increment updates the loop control variable after each iteration.

Example

```
x++
```

This increases the value of x by 1.

Flow of Execution of for Loop

The for loop executes in the following order:

1. Initialization executes once.
2. Condition is checked.
3. If condition is true → loop body executes.
4. Increment executes.
5. Condition is checked again.
6. Steps repeat until condition becomes false.

Example: Print Numbers 1 to 100**Program**

```
#include <stdio.h>
int main(void)
{
    int x;
    for(x = 1; x <= 100; x++)
        printf("%d ", x);

    return 0;
}
```

Explanation

Step-by-step execution:

1 Initialization

x = 1

2 Condition check

1 <= 100 → true

3 Print

1

4 Increment

x = 2

This process continues until:

x = 101

Condition becomes false

101 <= 100 → false

Loop stops.

Example: Loop with Multiple Statements

```
for(x = 100; x != 65; x -= 5)
{
    z = x * x;
```

```
    printf("The square of %d is %d", x, z);  
}
```

Explanation

Initial value

$x = 100$

Each iteration

$x = x - 5$

Values of x

100

95

90

85

80

75

70

Loop stops when

$x = 65$

Important Property of for Loop

The **condition is checked before executing the loop body**.

Therefore the loop may execute **zero times**.

Example

$x = 10$

$y = 10$

Loop

```
for(y=10; y!=x; y++)
```

```
printf("%d", y);
```

Condition

$10 \neq 10 \rightarrow \text{false}$

Loop body never executes.

Variations of the for Loop

The for loop in C allows many variations.

Multiple Loop Variables

Using the **comma operator**, multiple variables can control a loop.

Example

```
for(x=0, y=0; x+y < 10; x++)
{
    y = getchar();
    y = y - '0';
}
```

Explanation

Two variables control the loop:

x

y

Initialization

x = 0

y = 0

Termination condition

x + y < 10

Example Program: Converging String Copy

This program copies characters from both ends of a string.

```
void converge(char *targ, char *src)
{
    int i, j;

    for(i=0, j=strlen(src); i<=j; i++, j--)
    {
        targ[i] = src[i];
        targ[j] = src[j];
    }
}
```

Working

Two variables move toward each other:

$i \rightarrow$ start of string

$j \rightarrow$ end of string

Each iteration:

i increases

j decreases

When

$i > j$

loop stops.

Using Complex Conditions

The condition can be any relational or logical expression.

Example

```
for(x=0; x<3 && strcmp(str,"password"); x++)
```

This loop stops when:

1 Correct password entered OR

2 Three attempts finished

Example: Login System

```
void sign_on(void)
{
    char str[20];
    int x;

    for(x=0; x<3 && strcmp(str,"password"); x++)
    {
        printf("Enter password:");
        gets(str);
    }

    if(x==3)
        return;
}
```

Unusual Use of for Loop

The three sections of a for loop may contain **any valid expressions**.

Example

```
for(prompt(); t = readnum(); prompt())  
    sqrnum(t);
```

Initialization

prompt()

Condition

t = readnum()

Increment

prompt()

Missing Expressions in for Loop

All three expressions are optional.

Example

```
for(x=0; x!=123; )  
    scanf("%d",&x);
```

Only the condition exists.

Infinite Loop

An infinite loop runs forever.

Example

```
for(;;)  
    printf("This runs forever");
```

Because the condition is missing.

Missing condition means

true

Infinite Loop with Break

```
for(;;)
{
    ch = getchar();

    if(ch == 'A')
        break;
}
```

Loop stops when **A** is entered.

Empty Body Loop

Loops may have **no body**.

Example

```
for(; *str==' '; str++);
```

Purpose:

Skip leading spaces.

Delay Loop

```
for(t=0; t<10000; t++);
```

Used to create **time delay**.

Note: Some compilers remove such loops during optimization.

Declaring Variables inside for Loop

In **C99 and C++**, variables can be declared inside the loop.

Example

```
for(int i=0; i<10; i++)
j = i * i;
```

Scope of **i** is limited to the loop.

This feature is **not supported in C89**.

The while Loop

The while loop repeatedly executes a block of code **as long as a condition remains true**.

Syntax

```
while(condition)
statement;
```

Or

```
while(condition)
{
    statements
}
```

Working

- 1 Condition is checked.
- 2 If true → loop body executes.
- 3 Condition checked again.
- 4 Loop continues until condition becomes false.

Example: Wait for Character

```
char wait_for_char()
{
    char ch;

    ch = '\0';

    while(ch != 'A')
        ch = getchar();

    return ch;
}
```

Loop continues until user types A.

Example: Padding a String

```
void pad(char *s, int length)
{
```

```
int l;  
  
l = strlen(s);  
  
while(l < length)  
{  
    s[l] = ' ';  
    l++;  
}  
  
s[l] = '\0';  
}
```

This function **adds spaces** until the string reaches a specified length.

Example with Multiple Conditions

```
while(working)  
{  
    working = process1();  
  
    if(working)  
        working = process2();  
  
    if(working)  
        working = process3();  
}
```

Any function returning false stops the loop.

Empty while Loop

```
while((ch = getchar()) != 'A');
```

Loop runs until **A** is typed.

The do-while Loop

Unlike for and while, the **do-while** loop tests the condition after executing the loop body.

Therefore the loop body **always executes at least once**.

Syntax

```
do
{
    statements
}
while(condition);
```

Note

semicolon is required

Working

- 1 Loop body executes first.
- 2 Condition is tested.
- 3 If condition true → loop repeats.
- 4 If condition false → loop stops.

Example

```
do
{
    scanf("%d",&num);
}
while(num > 100);
```

Loop continues until the number is **100 or less**.

Menu Program Using do-while

```
void menu()
{
    char ch;

    printf("1.Check Spelling\n");
    printf("2.Correct Errors\n");
    printf("3.Display Errors\n");
```

```

do
{
    ch = getchar();

    switch(ch)
    {
        case '1':
            check_spelling();
            break;

        case '2':
            correct_errors();
            break;

        case '3':
            display_errors();
            break;
    }
}
while(ch!='1' && ch!='2' && ch!='3');
}

```

Loop continues until a **valid option** is selected.

Difference Between **for**, **while**, and **do-while**

Feature	for	while	do-while
Condition check	Beginning	Beginning	End
Minimum iterations	0	0	1
Best use	Known repetitions	Unknown repetitions	Menu driven programs

Iteration statements are essential constructs in C programming that allow repeated execution of instructions. C provides three loops—**for**, **while**, and **do-while**—each designed for different programming situations. The for loop is most suitable when the number of iterations is known, while the while and do-while loops are useful when the termination condition is based on runtime events. Proper understanding of iteration statements enables programmers to design efficient algorithms and avoid redundant code.

JUMP STATEMENTS IN C

In C programming, **jump statements** are used to **transfer program control from one part of the program to another immediately**.

Normally, program execution follows a **sequential order**, meaning statements execute one after another. However, sometimes it becomes necessary to **change the normal flow of execution**, such as:

- exiting a loop early
- skipping certain statements
- returning from a function
- terminating the entire program

For such situations, C provides **jump statements**.

Jump statements are statements that cause an unconditional transfer of control to another part of the program.

They override the normal sequential flow of execution.

Types of Jump Statements in C

C provides **four jump statements**:

1. **return**
2. **goto**
3. **break**
4. **continue**

Each of these performs a different type of control transfer.

Statement Purpose

return	Terminates a function and returns control to the caller
goto	Transfers control to a labeled statement
break	Immediately terminates a loop or switch
continue	Skips remaining loop statements and starts next iteration

1 The return Statement

The **return statement** is used to **exit from a function and return control to the function that called it**.

It is categorized as a jump statement because it **causes program execution to jump back to the point where the function was called.**

Syntax

Returning a value

return expression;

Returning without a value

return;

Working of return

When the return statement is executed:

1. The current function **stops execution immediately.**
2. Control **returns to the calling function.**
3. If an expression is present, **its value becomes the return value of the function.**

Example: Returning a Value

```
#include <stdio.h>
int square(int x)
{
    return x * x;
}

int main()
{
    int result;

    result = square(5);

    printf("Square = %d", result);

    return 0;
}
```

Explanation

Function call

square(5)

Execution

return 5 * 5

Return value

25

Output

Square = 25

Returning Without Value

If a function is declared as **void**, it does not return any value.

Example

```
void message()
{
    printf("Hello");
    return;
}
```

Important Rules of return

Rule 1

A function with **non-void return type must return a value.**

Example

```
int func()
{
    return 5;
}
```

Rule 2

A **void function cannot return a value.**

Incorrect example

```
void func()
{
    return 10; // error
}
```

Rule 3

A function may contain **multiple return statements**.

Example

```
int abs_value(int x)
{
    if(x >= 0)
        return x;
    else
        return -x;
}
```

Rule 4

When a return statement executes, the **function stops immediately**.

Rule 5

Reaching the closing brace of a function behaves like

`return;`

If the function has a return type, the returned value becomes **undefined**.

C89 vs C99 Behavior

C89

A function declared as returning a value **may omit return**.

But this returns a **garbage value**.

C99

A **return value is mandatory** for non-void functions.

2 The goto Statement

The **goto statement** transfers control directly to another statement labeled in the program.

It is a **direct jump instruction**.

Syntax

```
goto label;
```

```
...
```

```
label:  
statement;
```

Important Rules

1. A label is **an identifier followed by a colon**

Example

```
loop:
```

2. The label must be **inside the same function**
3. goto **cannot jump between functions**

Example: Loop Using goto

```
#include <stdio.h>
```

```
int main()  
{  
    int x = 1;
```

```
loop1:
```

```
    printf("%d ", x);  
    x++;
```

```
    if(x <= 10)  
        goto loop1;
```

```
    return 0;  
}
```

Output

```
1 2 3 4 5 6 7 8 9 10
```

Disadvantages of goto

Many programmers avoid goto because it can create **spaghetti code**.

Problems include:

- difficult debugging
- poor readability
- confusing program flow

When goto May Be Useful

Although rarely used, goto can help in:

- exiting deeply nested loops
- complex error handling
- breaking out of multiple conditions

3 The break Statement

The **break statement** causes **immediate termination of a loop or switch statement**.

It forces program control to move to the **first statement after the loop or switch**.

Syntax

```
break;
```

break in Loops

When break executes inside a loop:

1. The loop stops immediately.
2. Control moves to the next statement after the loop.

Example

```
#include <stdio.h>
int main()
{
    int t;
    for(t = 0; t < 100; t++)
    {
        printf("%d ", t);
        if(t == 10)
            break;
    }
}
```

```
    return 0;  
}
```

Output

0 1 2 3 4 5 6 7 8 9 10

Loop stops when **t = 10**.

Example: break in Nested Loops

```
for(t=0; t<100; t++)  
{  
    count = 1;  
    for(;;)  
    {  
        printf("%d ", count);  
        count++;  
        if(count == 10)  
            break;  
    }  
}
```

Explanation

The break exits only the **inner loop**.

Outer loop continues.

Output

1 2 3 4 5 6 7 8 9

repeated **100 times**.

break in switch

In switch statements, break prevents **fall-through**.

Example

```
switch(x)  
{  
    case 1:  
        printf("One");  
        break;  
    case 2:  
        printf("Two");  
}
```

```
    break;
}
```

4 The exit() Function

Although not technically a jump statement, exit() **terminates the entire program.**

Syntax

```
void exit(int return_code);
```

Header file required

```
#include <stdlib.h>
```

Example

```
#include <stdlib.h>
```

```
int main()
{
    if(!check_system())
        exit(1);

    run_program();
}
```

Meaning of Return Codes

Code	Meaning
0	Normal termination
Non-zero	Error condition

Common macros:

```
EXIT_SUCCESS
```

```
EXIT_FAILURE
```

```
]
```

Example: Exit from Menu

```
case '4':
```

```
    exit(0);
```

Program terminates immediately.

5 The continue Statement

The **continue** statement skips the remaining statements in the loop and begins the next iteration immediately.

Unlike break, continue **does not terminate the loop**.

Syntax

```
continue;
```

Behavior in Different Loops

Loop Type Behavior

for Executes increment and condition test

while Returns to condition check

do-while Goes to condition check

Example: Counting Spaces

```
#include <stdio.h>
int main()
{
    char s[80], *str;
    int space;

    printf("Enter a string: ");
    gets(s);
    str = s;
    for(space=0; *str; str++)
    {
        if(*str != ' ')
            continue;

        space++;
    }
    printf("%d spaces\n", space);
    return 0;
}
```

Explanation

The loop checks each character.

If the character **is not a space**, continue skips the increment statement.

Only spaces are counted.

Example: Character Coding Program

```
void code()
{
    char done, ch;

    done = 0;

    while(!done)
    {
        ch = getchar();

        if(ch == '$')
        {
            done = 1;
            continue;
        }

        putchar(ch + 1);
    }
}
```

How It Works

Each character is shifted forward.

Example

A → B

B → C

C → D

Typing \$ terminates the loop.

Difference Between break and continue

Feature	break	continue
Effect	Terminates loop	Skips remaining statements
Loop continues	No	Yes
Used in	loops and switch loops only	

Jump statements provide powerful mechanisms for controlling the flow of execution in C programs. They allow programmers to exit functions, terminate loops, skip unnecessary operations, and even terminate entire programs when required. The four jump statements—return, goto, break, and continue—serve different purposes and must be used carefully to maintain readability and logical structure of the program.

EXPRESSION STATEMENTS AND BLOCK STATEMENTS IN C

In C programming, every program consists of **statements** that specify actions to be performed. Among these, **expression statements** and **block statements** play an important role in structuring programs and organizing logic.

Although expressions are studied in detail in earlier chapters, understanding **how expressions become statements and how statements can be grouped into blocks** is essential for writing structured C programs.

Expression Statements

An **expression statement** is the simplest type of statement in C.

It consists of:

- a **valid expression**
- followed by a **semicolon (;)**

The semicolon indicates the **end of the statement**.

An expression statement is a statement formed by writing a valid expression followed by a semicolon.

General Syntax

expression;

Example

$x = y + 5;$

Here:

- $x = y + 5$ is the expression
- $;$ terminates the statement

How Expression Statements Work

When an expression statement executes:

1. The expression is evaluated.

2. Any **side effects** (such as assignment or function calls) occur.
3. Control moves to the next statement.

Common Types of Expression Statements

Expression statements usually perform operations such as:

- **assignment**
- **function calls**
- **arithmetic operations**
- **logical operations**
- **increment/decrement operations**

Examples of Expression Statements

Example 1: Function Call

```
func();
```

Explanation

- `func()` is a function call.
- When this statement executes, the function `func()` is called.

Function calls are one of the **most common expression statements**.

Example 2: Assignment Statement

```
a = b + c;
```

Explanation

1. The expression `b + c` is evaluated.
2. The result is assigned to `a`.

Example

```
a = 5 + 3;
```

Result

```
a = 8
```

Example 3: Expression with Function Call

```
b + f();
```

This expression appears unusual but is still valid.

Explanation

- The compiler evaluates $b + f()$.
- The function $f()$ may perform an important operation.
- Even if the result of the expression is not stored, the function still executes.

Example

```
int f()
{
    printf("Function executed\n");
    return 10;
}
int main()
{
    int b = 5;
    b + f();
}
```

Output

Function executed

Even though the result of $b + f()$ is unused, the function still executes.

Empty Statement (Null Statement)

An expression statement may also be **empty**.

Example

```
;
```

This is called a **null statement**.

A **null statement** is a statement consisting only of a semicolon.

It performs **no operation**.

Uses of Null Statements

Null statements are useful in certain programming situations.

Example

```
while(getchar() != '\n');
```

Explanation

- The loop continues reading characters until a newline is found.
- The loop body is empty.

- The semicolon represents a null statement.

Example: Using Null Statement in a Loop

```
#include <stdio.h>
```

```
int main()
{
    int i = 0;

    while(i < 5)
        i++;

    printf("%d", i);

    return 0;
}
```

Output

5

BLOCK STATEMENTS

Often, a program needs to **execute multiple statements together as a single unit**.

To achieve this, C provides **block statements**.

A block groups several statements so they behave like **one single statement**.

A **block statement** is a group of statements enclosed within curly braces { } that are treated as a single unit.

Block statements are also called **compound statements**.

Syntax of Block Statement

```
{
    statement1;
    statement2;
    statement3;
}
```

All statements inside the braces are treated as **one statement**.

Example of Block Statement

```
{
    int a = 10;
    int b = 20;
```

```
    printf("%d", a + b);  
}
```

This block contains three statements.

Purpose of Block Statements

Block statements are commonly used to:

1. **Group related statements**
2. **Create multi-statement bodies for control structures**
3. **Improve readability of programs**

Block Statements in Control Structures

Block statements are frequently used with statements like:

- if
- for
- while
- switch

Example

```
if(x > 10)  
{  
    printf("Greater than 10\n");  
    printf("Condition satisfied\n");  
}
```

Without block statements, only **one statement could belong to the if.**

Example Without Block Statement

```
if(x > 10)  
    printf("Greater than 10\n");  
    printf("Condition satisfied\n");
```

In this case:

Only the first statement belongs to if.

The second statement executes regardless of the condition.

Free-Standing Block Statement

A block statement can appear **anywhere a statement is allowed**, even without control structures.

Example

```
#include <stdio.h>
int main(void)
{
    int i;

    { /* free-standing block */
        i = 120;
        printf("%d", i);
    }
    return 0;
}
```

Explanation

The block

```
{
    i = 120;
    printf("%d", i);
}
```

is executed like a single statement.

Output

120

Advantages of Block Statements

Block statements provide several benefits:

1 Better Program Structure

Grouping statements makes programs easier to understand.

2 Logical Organization

Related operations can be grouped together.

3 Multiple Statements in Control Structures

Blocks allow multiple statements inside loops or conditions.

Example

```
for(i=0;i<5;i++)
{
    printf("%d\n", i);
    printf("Loop running\n");
}
```

Scope of Variables in Blocks

Variables declared inside a block are **local to that block**.

Example

```
{  
    int x = 5;  
}
```

Outside the block:

x is not accessible

Nested Blocks

Blocks can also be **nested inside other blocks**.

Example

```
{  
    int a = 5;  
    {  
        int b = 10;  
        printf("%d", a + b);  
    }  
}
```

Key Differences Between Expression Statements and Block Statements

Feature	Expression Statement	Block Statement
Definition	Single expression followed by semicolon	Group of statements enclosed in { }
Purpose	Performs a specific operation	Groups statements as one unit
Example	a = b + c;	{ a = 5; b = 10; }
Statements	Usually single	Multiple statements

Expression statements and block statements form fundamental components of C programs. An expression statement represents a valid expression followed by a semicolon and performs operations such as assignments or function calls. Block statements, also known as compound statements, group multiple statements together and treat them as a single unit. They play a vital role in organizing code, controlling program flow, and improving readability and maintainability of programs.

Thank You

We're glad to be part of your engineering journey.
Keep exploring, keep innovating, and keep growing.

AZ Documents

Your Engineering Study Partner

www.azdocuments.in

Quality notes, simplified explanations, and
student-focused resources for every semester.

Connect With Us:

 www.azdocuments.in [[Click Here](#)]

 [@azdocuments](#) [[Click Here](#)]

 Join Our Student Community: [[Click Here](#)]

© 2025 AZ Documents. All rights reserved.

This material is intended for educational purposes only.
Redistribution without permission is prohibited.